

Session 1 - Lab Programs

Name: H ASRAAR AHMED

Course name: DAA

Reg no: 192524028

Course code: CSA1513

Program 1

Question

Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1: words = ["abc","car","ada","racecar","cool"] -> Output: "ada" Example 2: words = ["notapalindrome","racecar"] -> Output: "racecar"

Program Code

```
#include <stdio.h>
#include <string.h>

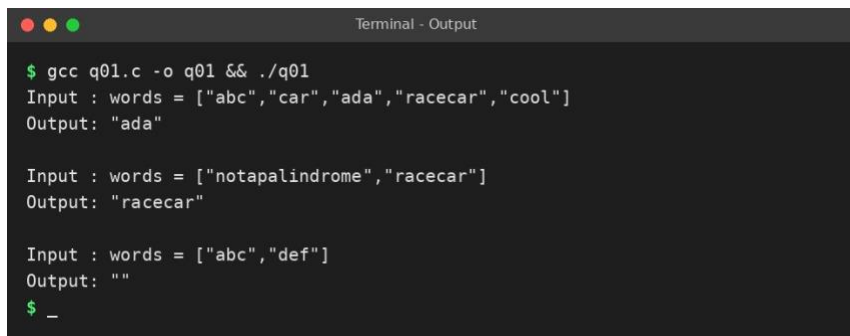
/* Returns 1 if the string reads the same forward and backward
*/ int isPalindrome(const char *s) {      int i = 0, j =
(int)strlen(s) - 1;      while (i < j) {          if (s[i] !=
s[j]) return 0;          i++;          j--;          }      return 1; }

/* Returns the first palindromic string, or "" if none exists
*/ const char *firstPalindrome(const char *words[], int n) {
for (int i = 0; i < n; i++)          if
(isPalindrome(words[i])) return words[i];      return ""; }

void run(const char *words[], int n) {      printf("Input :
words = [");      for (int i = 0; i < n; i++)
printf("\'%s\'\'%s\'", words[i], i < n - 1 ? ", " : " ");
printf("]\n");      printf("Output: \'%s\'\\n\\n",
firstPalindrome(words, n)); }

int main() {      const char *w1[] = {"abc", "car", "ada",
"racecar", "cool"};      const char *w2[] = {"notapalindrome",
"racecar"};      const char *w3[] = {"abc", "def"};      run(w1,
5);      run(w2, 2);      run(w3, 2);      return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q01.c -o q01 && ./q01
Input : words = ["abc","car","ada","racecar","cool"]
Output: "ada"

Input : words = ["notapalindrome","racecar"]
Output: "racecar"

Input : words = ["abc","def"]
Output: ""

$ _
```

You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate: answer1 = the number of indices i such that nums1[i] exists in nums2; answer2 = the number of indices i such that nums2[i] exists in nums1. Return [answer1, answer2].

Example 1: nums1 = [2,3,2], nums2 = [1,2] -> Output: [2,1] Example 2: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6] -> Output: [3,4]

Program Code

```
#include <stdio.h>

/* Check whether x is present in arr[0..n-1]
*/ int exists(int arr[], int n, int x) {
for (int i = 0; i < n; i++)          if
(arr[i] == x) return 1;      return 0; }

void solve(int nums1[], int n, int nums2[], int m)
{      int answer1 = 0, answer2 = 0;      for (int i
= 0; i < n; i++)          if (exists(nums2, m,
nums1[i])) answer1++;      for (int i = 0; i < m;
```

Program 2

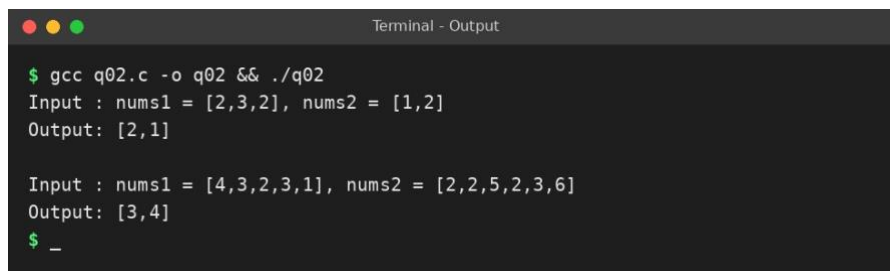
Question

```
i++)          if (exists(nums1, n, nums2[i]))
answer2++;

    printf("Input : nums1 = [");    for (int i = 0; i < n; i++)
printf("%d%s", nums1[i], i < n - 1 ? "," : "");    printf("], nums2 = [");
for (int i = 0; i < m; i++) printf("%d%s", nums2[i], i < m - 1 ? "," : "");
printf("]\nOutput: [%d,%d]\n\n", answer1, answer2); }

int main() {    int a1[] = {2, 3, 2},    b1[] = {1,
2};    int a2[] = {4, 3, 2, 3, 1}, b2[] = {2, 2, 5, 2,
3, 6};    solve(a1, 3, b1, 2);    solve(a2, 5, b2, 6);
return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q02.c -o q02 && ./q02
Input : nums1 = [2,3,2], nums2 = [1,2]
Output: [2,1]

Input : nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]
Output: [3,4]
$ _
```

You are given a 0-indexed integer array `nums`. The distinct count of a subarray `nums[i..j]` is the number of distinct values in it. Return the sum of the squares of the distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1: `nums = [1,2,1]` -> Output: 15 *Example 2: `nums = [1,1]` -> Output: 3*

Program Code

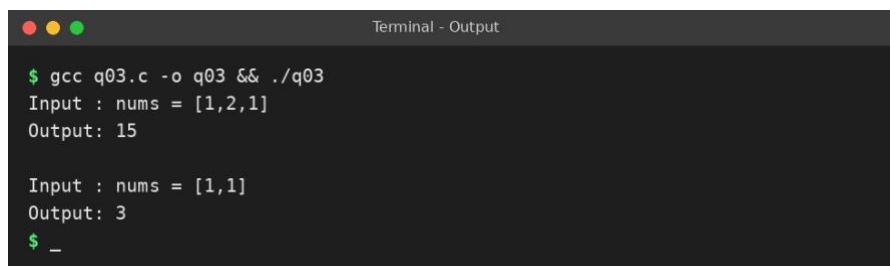
```
#include <stdio.h>
#include <string.h>

/* Assumes 1 <= nums[i] <= 100 (problem constraint) */ long long
sumOfSquares(int nums[], int n) {    long long total = 0;    for (int i
= 0; i < n; i++) {        int seen[101];        memset(seen, 0,
sizeof(seen));        int distinct = 0;        for (int j = i; j < n;
j++) {            /* extend subarray nums[i..j] */            if
(!seen[nums[j]]) {                seen[nums[j]] = 1;
distinct++;                total += (long long)distinct *
distinct;            }        }        return total;    }

void run(int nums[], int n) {    printf("Input : nums = [");    for (int i
= 0; i < n; i++) printf("%d%s", nums[i], i < n - 1 ? "," : "");
printf("]\nOutput: %lld\n\n", sumOfSquares(nums, n)); }

int main() {    int
a[] = {1, 2, 1};
int b[] = {1, 1};
run(a, 3);    run(b,
2);    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q03.c -o q03 && ./q03
Input : nums = [1,2,1]
Output: 15

Input : nums = [1,1]
Output: 3
$ _
```

Program 3

Question

Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

Example 1: `nums = [3,1,2,2,2,1,3]`, `k = 2` -> Output: 4 *Example 2: `nums = [1,2,3,4]`, `k = 1` -> Output: 0*

Program Code

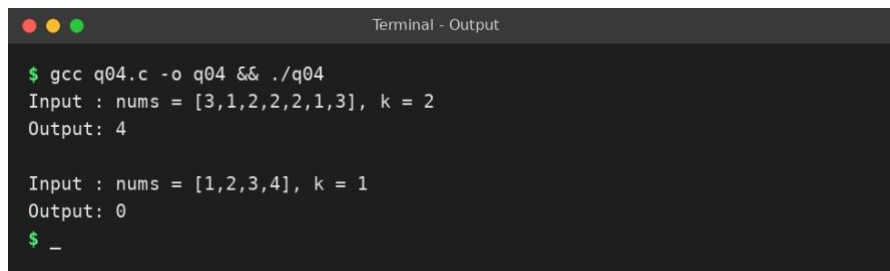
```
#include <stdio.h>

int countPairs(int nums[], int n, int k) {    int count = 0;
for (int i = 0; i < n; i++)                for (int j = i + 1; j < n;
j++)    if (nums[i] == nums[j] && ((long long)i * j) %
k == 0)    count++;    return count; }

void run(int nums[], int n, int k) {
printf("Input : nums = [");
    for (int i = 0; i < n; i++) printf("%d%s", nums[i], i < n - 1 ? ", " :
 "");    printf("], k = %d\nOutput: %d\n\n", k, countPairs(nums, n, k)); }

int main() {    int a[] = {3, 1,
2, 2, 2, 1, 3};    int b[] = {1,
2, 3, 4};    run(a, 7, 2);
run(b, 4, 1);    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q04.c -o q04 && ./q04
Input : nums = [3,1,2,2,2,1,3], k = 2
Output: 4

Input : nums = [1,2,3,4], k = 1
Output: 0
$ _
```

Write a program for the below test cases with the least time complexity (find the maximum element).

Input: {1, 2, 3, 4, 5} Expected Output: 5 *Input: {7, 7, 7, 7, 7} Expected Output: 7*

Input: {-10, 2, 3, -4, 5} Expected Output: 5

Program Code

```
#include <stdio.h>

/* Single pass: Time O(n), Space O(1) -- the least possible time complexity
*/ int findMax(int arr[], int n) {    int max = arr[0];    for (int i = 1;
i < n; i++)    if (arr[i] > max) max = arr[i];    return max; }

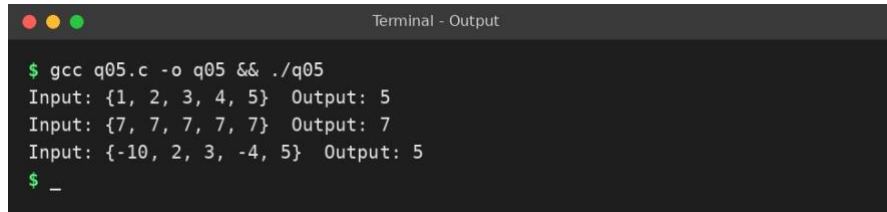
void run(int arr[], int n) {    printf("Input: {");    for (int i = 0; i <
n; i++) printf("%d%s", arr[i], i < n - 1 ? ", " : "");    printf("}
Output: %d\n", findMax(arr, n)); }

int main() {    int t1[] = {1,
2, 3, 4, 5};    int t2[] = {7,
7, 7, 7, 7};    int t3[] = {-10,
2, 3, -4, 5};    run(t1, 5);
run(t2, 5);    run(t3, 5);
return 0; }
```

Program 4

Question

Output Screenshot



```
Terminal - Output
$ gcc q05.c -o q05 && ./q05
Input: {1, 2, 3, 4, 5} Output: 5
Input: {7, 7, 7, 7, 7} Output: 7
Input: {-10, 2, 3, -4, 5} Output: 5
$ _
```

You have an algorithm that processes a list of numbers. It first sorts the list using an efficient sorting algorithm and then finds the maximum element in the sorted list. Write the code for the same.

1. Empty list: Input [] -> None or an appropriate message indicating the list is empty 2. Single element list: Input [5] -> 5 3. All elements are the same: Input [3, 3, 3, 3, 3] -> 3

Program Code

```
#include <stdio.h>
#include <stdlib.h>

/* Merge sort : O(n log n) time */ void merge(int a[], int l, int m,
int r) {
    int n1 = m - l + 1, n2 = r - m;
    int *L = malloc(n1 * sizeof(int)), *R = malloc(n2 * sizeof(int));
    for (int i = 0; i < n1; i++) L[i] = a[l + i];
    for (int j = 0; j < n2; j++) R[j] = a[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
    free(L);
    free(R);
}

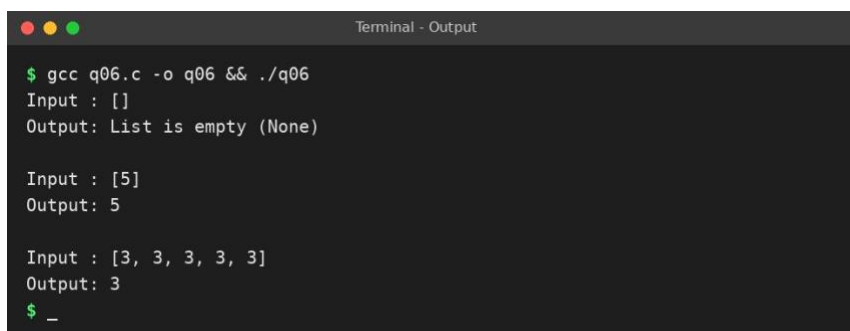
void mergeSort(int a[], int l, int r)
{
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}

void run(int a[], int n) {
    printf("Input : [");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i < n - 1 ? ", " : "");
    printf("]\n");
    if (n == 0) {
        printf("Output: List is empty (None)\n\n");
        return;
    }
    mergeSort(a, 0, n - 1);
    printf("Output: %d\n\n", a[n - 1]);
}

int main() {
    int t1[1] = {0};
    /* empty list */
    int t2[] = {5};
    int t3[] = {3, 3, 3, 3, 3};
    run(t1, 0);
    run(t2, 1);
    run(t3, 5);
    return 0;
}
```

Output Screenshot

Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?



```
Terminal - Output
$ gcc q06.c -o q06 && ./q06
Input : []
Output: List is empty (None)

Input : [5]
Output: 5

Input : [3, 3, 3, 3, 3]
Output: 3
$ _
```

Program 5

Question

1. Some duplicate elements: [3, 7, 3, 5, 2, 5, 9, 2] -> [3, 7, 5, 2, 9] 2. Negative and positive numbers: [-1, 2, -1, 3, 2, -2] -> [-1, 2, 3, -2] 3. Large numbers: [1000000, 999999, 1000000] -> [1000000, 999999] (Order may vary based on the algorithm used)

Program Code

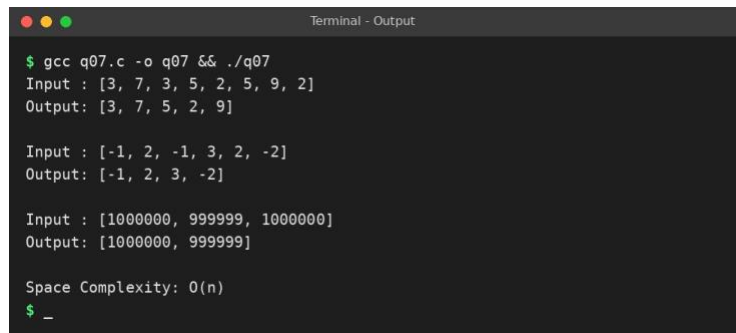
```
#include <stdio.h>
#include <stdlib.h>

/* Hash set with linear probing: Time O(n) average, Space O(n)
*/ int uniqueElements(int a[], int n, int out[]) {    int cap
= 2 * n + 1;    int *table = malloc(cap * sizeof(int));
int *used = calloc(cap, sizeof(int));    int k = 0;    for
(int i = 0; i < n; i++) {
    long h = ((long)a[i] % cap + cap) % cap;
    while (used[h] && table[h] != a[i]) h = (h + 1) % cap;
if (!used[h]) {        /* first time we see this value */
used[h] = 1;        table[h] = a[i];        out[k++] = a[i];
}    }    free(table);    free(used);    return k; }

void run(int a[], int n) {    int out[100];    int k = uniqueElements(a,
n, out);    printf("Input : [");    for (int i = 0; i < n; i++)
printf("%d%s", a[i], i < n - 1 ? ", " : "");    printf("]\nOutput: [");
for (int i = 0; i < k; i++) printf("%d%s", out[i], i < k - 1 ? ", " : "");
printf("]\n\n"); }

int main() {
    int t1[] = {3, 7, 3, 5, 2, 5, 9, 2};
int t2[] = {-1, 2, -1, 3, 2, -2};    int
t3[] = {1000000, 999999, 1000000};
run(t1, 8);    run(t2, 6);    run(t3,
3);    printf("Space Complexity:
O(n)\n");    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q07.c -o q07 && ./q07
Input : [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]

Input : [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]

Input : [1000000, 999999, 1000000]
Output: [1000000, 999999]

Space Complexity: O(n)
$ _
```

Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code.

Program Code

```
#include <stdio.h>

/* Bubble sort: Best O(n) (already sorted), Average/Worst O(n^2), Space O(1)
*/ void bubbleSort(int a[], int n) {    for (int i = 0; i < n - 1; i++) {
int swapped = 0;        for (int j = 0; j < n - i - 1; j++) {            if
(a[j] > a[j + 1]) {                int t = a[j];                a[j] = a[j
+ 1];                a[j + 1] = t;                swapped = 1;
}            }        }        if (!swapped) break;        /* array already sorted */
} }

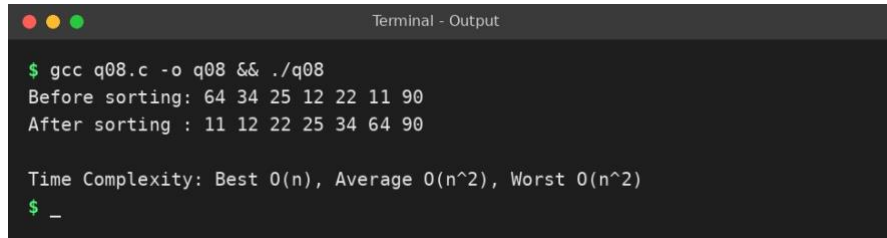
void print(const char *label, int a[], int n) {
printf("%s", label);    for (int i = 0; i < n;
i++) printf("%d ", a[i]);    printf("\n"); }
```

Program 6

Question

```
int main() {    int a[] = {64, 34, 25, 12, 22, 11, 90};    int n =
sizeof(a) / sizeof(a[0]);    print("Before sorting: ", a, n);
bubbleSort(a, n);    print("After sorting : ", a, n);    printf("\nTime
Complexity: Best O(n), Average O(n^2), Worst O(n^2)\n");    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q08.c -o q08 && ./q08
Before sorting: 64 34 25 12 22 11 90
After sorting : 11 12 22 25 34 64 90

Time Complexity: Best O(n), Average O(n^2), Worst O(n^2)
$ _
```

Check whether a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Example: $X = \{3, 4, 6, -9, 10, 8, 9, 30\}$, $KEY = 10 \rightarrow$ Element 10 is found at position 5
Example: $X = \{3, 4, 6, -9, 10, 8, 9, 30\}$, $KEY = 100 \rightarrow$ Element 100 is not found

Program Code

```
#include <stdio.h>

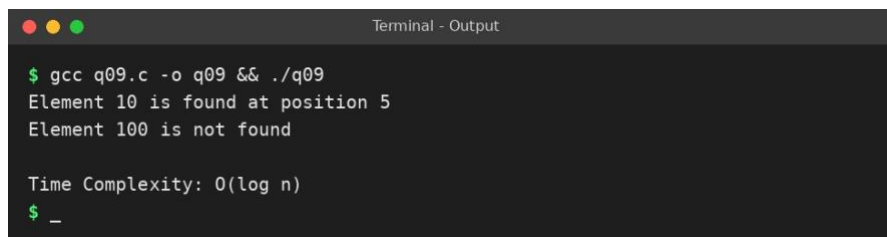
/* The sample array is not in sorted order, so we sort (value, original
index) pairs first, then binary search on the sorted values and report
the ORIGINAL 1-based position of the key. */ void sortWithIndex(int
val[], int idx[], int n) {    for (int i = 1; i < n; i++) {    /*
insertion sort on pairs */    int v = val[i], x = idx[i], j = i - 1;
while (j >= 0 && val[j] > v) {    val[j + 1] = val[j];
idx[j + 1] = idx[j];    j--;    }    val[j + 1] = v;
idx[j + 1] = x;    }
}

/* Binary search: Time O(log n), Space O(1). Returns index or -1
*/ int binarySearch(int arr[], int n, int key) {    int low = 0,
high = n - 1;    while (low <= high) {    int mid = low +
(high - low) / 2;    if (arr[mid] == key) return mid;
else if (arr[mid] < key) low = mid + 1;    else high = mid -
1;    }    return -1; }

void search(int X[], int n, int key) {    int val[20], idx[20];    for (int i =
0; i < n; i++) { val[i] = X[i]; idx[i] = i; }    sortWithIndex(val, idx, n);
int pos = binarySearch(val, n, key);    if (pos != -1) printf("Element %d is
found at position %d\n", key, idx[pos] + 1);    else    printf("Element %d
is not found\n", key); }

int main() {    int X[] = {3, 4, 6, -9,
10, 8, 9, 30};    int n = sizeof(X) /
sizeof(X[0]);    search(X, n, 10);
search(X, n, 100);    printf("\nTime
Complexity: O(log n)\n");    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q09.c -o q09 && ./q09
Element 10 is found at position 5
Element 100 is not found

Time Complexity: O(log n)
$ _
```

Program 7

Question

Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log n)$ time complexity and with the smallest space complexity possible.

Program Code

```
#include <stdio.h>

/* Heap sort: Time  $O(n \log n)$  in all cases, Space  $O(1)$ , no built-in functions
*/ void swap(int *a, int *b) {    int t = *a;    *a = *b;
    *b = t;
}

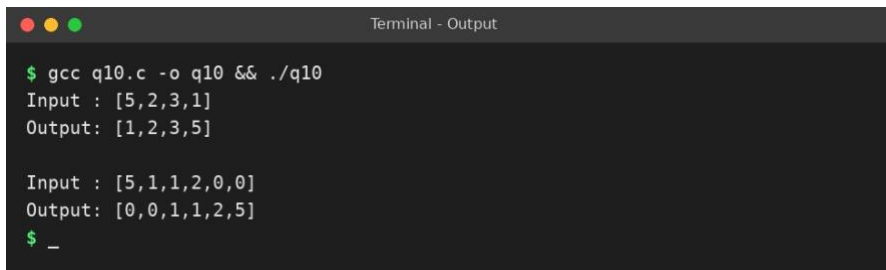
void heapify(int a[], int n, int i) {    int
largest = i, l = 2 * i + 1, r = 2 * i + 2;
if (l < n && a[l] > a[largest]) largest = l;
if (r < n && a[r] > a[largest]) largest = r;
if (largest != i) {        swap(&a[i],
&a[largest]);        heapify(a, n, largest);
} }

void sortArray(int a[], int n) {    for (int i = n / 2 - 1; i >=
0; i--)    /* build max heap */        heapify(a, n, i);    for
(int i = n - 1; i > 0; i--) {        /* move max to the end */
swap(&a[0], &a[i]);        heapify(a, i, 0);    } }

void run(int a[], int n) {    printf("Input : [");    for (int i = 0; i
< n; i++) printf("%d%s", a[i], i < n - 1 ? "," : "");    sortArray(a,
n);    printf("]\nOutput: [");    for (int i = 0; i < n; i++)
printf("%d%s", a[i], i < n - 1 ? "," : "");    printf("]\n\n"); }

int main() {    int t1[] = {5,
2, 3, 1};    int t2[] = {5, 1,
1, 2, 0, 0};    run(t1, 4);
run(t2, 6);    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q10.c -o q10 && ./q10
Input : [5,2,3,1]
Output: [1,2,3,5]

Input : [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
$ _
```